JAMSORA

eCommerce Platform

shop.jamsora.com

Software Requirements Specification (SRS)

&

Technical Architecture Document

Version	1.0.0
Stack	Laravel 11 + MySQL + Tailwind CSS
Date	May 2026
Classification	Confidential

Table of Contents

1. Introduction & Project Overview

1.1 Purpose

This document serves as the complete Software Requirements Specification (SRS) and Technical Architecture Document for rebuilding the Jamsora eCommerce website (shop.jamsora.com) from WordPress to a fully custom Laravel-based platform. It provides detailed requirements, architecture decisions, database design, API specifications, and a development roadmap for all stakeholders.

1.2 Project Scope

The project involves a complete rebuild of the existing WordPress eCommerce site into a modern, scalable, high-performance Laravel application with:

- A fully custom Content Management System (CMS) for non-technical administrators
- A high-performance, SEO-optimised storefront
- A secure multi-role authentication system
- A comprehensive order management and fulfilment pipeline
- Professional transactional email system
- RESTful API-ready architecture for future mobile apps

1.3 Technology Stack

Layer	Technology	Purpose
Backend Framework	Laravel 11 (LTS)	Core application framework
Language	PHP 8.3+	Server-side language
Database	MySQL 8.0+	Primary relational database
Cache / Queue	Redis 7+	Session, cache, and queue backend
Search	Laravel Scout + Meilisearch	Full-text product search
Frontend	Blade + Tailwind CSS 3 + Alpine.js	Server-rendered UI
Build Tool	Vite	Asset bundling and HMR
Job Queue	Laravel Horizon (Redis)	Background jobs monitoring
Storage	Laravel Filesystem (local / S3)	Media and file storage
Email	Laravel Mail + SMTP / Mailgun	Transactional emails
Authentication	Laravel Breeze / Fortify	Auth scaffolding
Payments	Stripe / Razorpay / COD	Payment processing
Server	Nginx + PHP-FPM	Web server
Container	Docker (local dev)	Development environment

2. Project Architecture

2.1 Architectural Pattern

The platform follows a Layered MVC Architecture with Domain-Driven Design (DDD) influences, ensuring separation of concerns, testability, and maintainability at scale.

> *Pattern: Controller → Service Layer → Repository → Eloquent Model → Database. Business logic lives exclusively in Service classes, never in Controllers or Models.*

2.2 Folder Structure

Path	Purpose
app/Http/Controllers/	Thin controllers — only input validation & response
app/Http/Controllers/Admin/	All admin CMS controllers
app/Http/Controllers/Api/	API v1 controllers
app/Services/	Business logic (OrderService, CartService, etc.)
app/Repositories/	Data access abstraction over Eloquent
app/Models/	Eloquent models with relationships
app/Jobs/	Queued jobs (SendOrderEmail, ProcessImage, etc.)
app/Mail/	Mailable classes
app/Observers/	Model event observers
app/Policies/	Authorization policies per model
app/Events/ & app/Listeners/	Domain events and handlers
app/Http/Middleware/	Request middleware
app/Http/Requests/	Form request validation classes
app/Helpers/	Global helper functions
resources/views/	Blade templates
resources/views/emails/	Email templates
resources/views/admin/	Admin CMS views
resources/views/frontend/	Customer-facing views
database/migrations/	Versioned schema migrations
database/seeders/	Database seeders
routes/web.php	Web routes (frontend + admin)
routes/api.php	API v1 routes
config/	Application configuration files
public/	Compiled assets, images
storage/app/public/	User-uploaded media

3. Database Design

3.1 Entity Relationship Overview

The database is normalised to 3NF with strategic denormalisation on read-heavy tables. All tables use unsigned BIGINT primary keys, snake_case naming, and soft deletes where appropriate.

3.2 Complete Table Schema

3.2.1 User & Authentication Tables

Table	Key Columns	Notes
users	id, name, email, password, phone, avatar, email_verified_at, status, role_id	Core user table; polymorphic with profiles
roles	id, name, slug, description	super_admin, admin, manager, customer
permissions	id, name, slug, group	Granular permission definitions
role_permissions	role_id, permission_id	Pivot: roles ↔ permissions
user_permissions	user_id, permission_id, granted	User-level overrides
social_accounts	id, user_id, provider, provider_id, token	OAuth social logins
password_reset_tokens	email, token, created_at	Password reset
otp_verifications	id, identifier, otp, type, expires_at, verified_at	OTP verification

3.2.2 Product Catalogue Tables

Table	Key Columns	Notes
categories	id, parent_id, name, slug, description, image, meta_title, meta_desc, sort_order, status	Self-referencing nested categories
brands	id, name, slug, logo, description, meta_title, meta_desc, status	Product brands
products	id, category_id, brand_id, name, slug, sku, short_desc, description, price, sale_price, cost_price, stock_qty, manage_stock, weight, dimensions, status, is_featured, meta_*	Core product
product_images	id, product_id, path, alt_text, is_primary, sort_order	Multiple product images
attributes	id, name, slug, type (select/color/size)	Configurable attributes
attribute_values	id, attribute_id, value, slug, color_code	Attribute options

Table	Key Columns	Notes
product_attributes	id, product_id, attribute_id	Products ↔ attributes
product_variations	id, product_id, sku, price, sale_price, stock_qty, image_id, status	Product variants
variation_attribute_values	variation_id, attribute_value_id	Variant ↔ attribute values
tags	id, name, slug	Product tags
product_tags	product_id, tag_id	Pivot
related_products	product_id, related_product_id	Manual related products

3.2.3 Order & Cart Tables

Table	Key Columns	Notes
carts	id, session_id, user_id, ip_address, expires_at	Persistent cart (guest + user)
cart_items	id, cart_id, product_id, variation_id, quantity, unit_price, options (JSON)	Cart line items
orders	id, user_id, order_number, subtotal, discount, shipping_cost, tax, total, status, payment_status, payment_method, notes, ip_address, user_agent	Master order record
order_items	id, order_id, product_id, variation_id, product_name, sku, quantity, unit_price, total, options (JSON)	Snapshotted at order time
order_addresses	id, order_id, type (billing/shipping), name, phone, address, city, state, country, zip	Address snapshot
order_status_history	id, order_id, status, comment, created_by, created_at	Full status audit trail
shipments	id, order_id, carrier, tracking_number, shipped_at, estimated_delivery	Shipment tracking
invoices	id, order_id, invoice_number, issued_at, due_at, pdf_path	Invoice records

3.2.4 Customer & Address Tables

Table	Key Columns	Notes
user_profiles	id, user_id, date_of_birth, gender, newsletter_subscribed	Extended profile

Table	Key Columns	Notes
addresses	id, user_id, label, name, phone, address, city, state, country, zip, is_default	Address book
wishlists	id, user_id, name, is_public	Multiple wishlists per user
wishlist_items	id, wishlist_id, product_id, variation_id, added_at	Wishlist products
notifications	id, user_id, type, title, message, data (JSON), read_at	In-app notifications
reviews	id, product_id, user_id, order_id, rating, title, body, status, helpful_count	Product reviews
review_images	id, review_id, path	Review photos

3.2.5 Coupon & Promotions Tables

Table	Key Columns	Notes
coupons	id, code, type (percentage/fixed/free_shipping), value, min_order, max_discount, usage_limit, used_count, per_user_limit, starts_at, expires_at, status	Coupon definitions
coupon_usages	id, coupon_id, user_id, order_id, discount_amount, used_at	Usage tracking
coupon_products	coupon_id, product_id	Restrict coupon to products
coupon_categories	coupon_id, category_id	Restrict to categories

3.2.6 CMS & Blog Tables

Table	Key Columns	Notes
pages	id, title, slug, status, meta_* , created_by	CMS pages
page_sections	id, page_id, component_type, data (JSON), sort_order, status	Page builder blocks
blog_categories	id, name, slug, description, meta_*	Blog categories
blog_posts	id, category_id, user_id, title, slug, excerpt, body, image, status, published_at, meta_*	Blog posts
blog_tags	id, name, slug	Blog tags

Table	Key Columns	Notes
blog_post_tags	post_id, tag_id	Pivot
blog_comments	id, post_id, user_id, parent_id, body, status	Threaded comments
menus	id, name, location, status	Navigation menus
menu_items	id, menu_id, parent_id, label, url, type, reference_id, sort_order, target	Menu tree
sliders	id, name, status	Hero sliders
slider_slides	id, slider_id, title, subtitle, image, link, sort_order, status	Individual slides
testimonials	id, name, position, company, avatar, rating, content, status, sort_order	Testimonials
faqs	id, question, answer, category, sort_order, status	FAQ entries

3.2.7 Settings & Media Tables

Table	Key Columns	Notes
settings	id, group, key, value (TEXT), type	Key-value settings store
media	id, user_id, folder_id, name, original_name, path, disk, mime_type, size, width, height, alt, title, description	Media library
media_folders	id, parent_id, name, path	Folder hierarchy
shipping_zones	id, name, countries (JSON), status	Shipping regions
shipping_methods	id, zone_id, name, type, rate, min_order, max_weight, status	Rates per zone
tax_rates	id, name, country, state, rate, is_compound, priority	Tax rules
payment_methods	id, name, code, config (JSON), is_active	Payment gateway config
currencies	id, code, name, symbol, exchange_rate, is_default	Multi-currency support
activity_logs	id, user_id, action, model_type, model_id, old_values (JSON), new_values (JSON), ip, user_agent	Admin audit trail

Table	Key Columns	Notes
newsletter_subscribers	id, email, name, status, subscribed_at, token	Newsletter list
contact_messages	id, name, email, subject, message, status, replied_at	Contact form

3.3 Database Index Strategy

The following indexes are critical for query performance:

- products: INDEX(status, category_id), INDEX(slug), INDEX(is_featured), FULLTEXT(name, description)
- orders: INDEX(user_id, status), INDEX(order_number), INDEX(created_at)
- cart_items: INDEX(cart_id), INDEX(session_id)
- reviews: INDEX(product_id, status), INDEX(user_id)
- blog_posts: INDEX(status, published_at), INDEX(slug), FULLTEXT(title, body)
- activity_logs: INDEX(user_id), INDEX(model_type, model_id)

4. Models List

All Eloquent models are located in app/Models/ with full relationship definitions, casts, and observers.

Model	Table	Key Relationships & Notes
User	users	hasOne Profile, hasMany Orders, Addresses, Wishlists; belongsTo Role
Role	roles	hasMany Users; belongsToMany Permissions
Permission	permissions	belongsToMany Roles
SocialAccount	social_accounts	belongsTo User
Category	categories	hasMany Children (self), Products; belongsTo Parent (self)
Brand	brands	hasMany Products
Product	products	belongsTo Category, Brand; hasMany Images, Variations, Reviews; belongsToMany Attributes, Tags
ProductImage	product_images	belongsTo Product
Attribute	attributes	hasMany AttributeValues; belongsToMany Products
AttributeValue	attribute_values	belongsTo Attribute; belongsToMany Variations
ProductVariation	product_variations	belongsTo Product; belongsToMany AttributeValues
Tag	tags	belongsToMany Products, BlogPosts
Cart	carts	hasMany CartItems; belongsTo User (nullable)
CartItem	cart_items	belongsTo Cart, Product, ProductVariation
Order	orders	belongsTo User; hasMany OrderItems, OrderAddresses, Shipments, StatusHistory; hasOne Invoice
OrderItem	order_items	belongsTo Order, Product, ProductVariation
OrderAddress	order_addresses	belongsTo Order
OrderStatusHistory	order_status_history	belongsTo Order
Shipment	shipments	belongsTo Order
Invoice	invoices	belongsTo Order
Address	addresses	belongsTo User
Wishlist	wishlists	belongsTo User; hasMany WishlistItems
WishlistItem	wishlist_items	belongsTo Wishlist, Product
Review	reviews	belongsTo Product, User, Order; hasMany ReviewImages
Notification	notifications	belongsTo User
Coupon	coupons	hasMany CouponUsages; belongsToMany Products, Categories

Model	Table	Key Relationships & Notes
CouponUsage	coupon_usages	belongsTo Coupon, User, Order
Page	pages	hasMany PageSections
PageSection	page_sections	belongsTo Page
BlogPost	blog_posts	belongsTo BlogCategory, User; belongsToMany Tags; hasMany Comments
BlogCategory	blog_categories	hasMany BlogPosts
BlogComment	blog_comments	belongsTo BlogPost, User; hasMany Children (self)
Menu	menus	hasMany MenuItems
MenuItem	menu_items	belongsTo Menu; hasMany Children (self)
Slider	sliders	hasMany Slides
Slide	slider_slides	belongsTo Slider
Testimonial	testimonials	standalone
Faq	faqs	standalone
Media	media	belongsTo User, MediaFolder
MediaFolder	media_folders	hasMany Children (self), Media
Setting	settings	key-value store; accessed via Settings facade
ShippingZone	shipping_zones	hasMany ShippingMethods
ShippingMethod	shipping_methods	belongsTo ShippingZone
TaxRate	tax_rates	standalone
Currency	currencies	standalone
NewsletterSubscriber	newsletter_subscribers	standalone
ContactMessage	contact_messages	standalone
ActivityLog	activity_logs	belongsTo User; morphsTo any model

5. Controllers List

5.1 Frontend Controllers (app/Http/Controllers/)

Controller	Key Methods
HomeController	index() — Home page with featured products, sliders, blog posts
ProductController	index(), show(), compare(), search()
CategoryController	show() — filtered product grid with pagination
CartController	index(), add(), update(), remove(), clear()
CheckoutController	index(), process(), guestCheckout(), estimateShipping()
OrderController	index(), show(), invoice(), cancel()
WishlistController	index(), add(), remove(), move()
ReviewController	store(), update(), destroy(), helpful()
AccountController	dashboard(), profile(), updateProfile(), changePassword()
AddressController	index(), store(), update(), destroy(), setDefault()
NotificationController	index(), markRead(), markAllRead()
BlogController	index(), show(), category(), tag()
PageController	show() — Dynamic CMS pages
NewsletterController	subscribe(), unsubscribe()
ContactController	show(), submit()
SearchController	index() — Full-text search with filters
CouponController	apply(), remove()
AuthController	Handled by Laravel Fortify/Breeze + Social

5.2 Admin CMS Controllers (app/Http/Controllers/Admin/)

Controller	Key Methods
Admin\DashboardController	index() — KPIs, charts, recent activity
Admin\ProductController	index(), create(), store(), edit(), update(), destroy(), toggleStatus(), bulkAction()
Admin\CategoryController	Full CRUD + reorder(), tree()
Admin\BrandController	Full CRUD
Admin\AttributeController	Full CRUD + values CRUD
Admin\OrderController	index(), show(), updateStatus(), createShipment(), generateInvoice(), refund()

Controller	Key Methods
Admin\CustomerController	index(), show(), notes(), ban(), impersonate()
Admin\CouponController	Full CRUD + usageReport()
Admin\ReviewController	index(), approve(), reject(), destroy()
Admin\BlogController	Full CRUD for posts and categories
Admin\PageController	Full CRUD + sections CRUD (Page Builder)
Admin\MediaController	index(), upload(), destroy(), createFolder(), moveMedia()
Admin\SliderController	Full CRUD + slides CRUD
Admin\MenuController	Full CRUD + reorder()
Admin\TestimonialController	Full CRUD
Admin\FaqController	Full CRUD
Admin\ShippingController	zones CRUD, methods CRUD
Admin\TaxController	Full CRUD
Admin\SettingsController	index(), update() grouped by tab
Admin\ReportController	salesReport(), revenueReport(), productReport(), customerReport(), export()
Admin\ActivityLogController	index(), show()
Admin\SeoController	update() — Meta for any page/product/category
Admin\NewsletterController	index(), export(), sendCampaign()

6. Middleware List

Middleware	Class	Purpose
auth	Laravel built-in	Require authenticated session
auth:api	Laravel Sanctum	Require valid API token
guest	Laravel built-in	Redirect authenticated users
verified	Laravel built-in	Require email verification
role	App\Http\Middleware\CheckRole	RBAC role gate
permission	App\Http\Middleware\CheckPermission	Granular permission gate
admin	App\Http\Middleware\AdminMiddleware	Restrict to admin roles
throttle:api	Laravel built-in	API rate limiting
throttle:login	App\Http\Middleware\LoginRateLimiter	5 attempts / minute for login
cart.session	App\Http\Middleware\CartSession	Initialize/restore cart session
currency	App\Http\Middleware\SetCurrency	Set active currency from session
seo	App\Http\Middleware\InjectSeoMeta	Inject dynamic meta tags per route
maintenance	App\Http\Middleware\MaintenanceMode	Admin-configurable maintenance mode
force.https	App\Http\Middleware\ForceHttps	Redirect HTTP to HTTPS in production
track.activity	App\Http\Middleware\TrackUserActivity	Update last_active_at timestamp
cors	Laravel built-in	CORS headers for API routes

7. Authentication System

7.1 Authentication Architecture

The platform uses Laravel Fortify for headless authentication with Blade views. Sessions are stored in Redis for scalability. Social OAuth uses Laravel Socialite.

7.2 Authentication Flows

Registration Flow

1. User submits registration form (name, email, password, phone)
2. RegisteredUserController validates with RegisterRequest (unique email, password rules)
3. UserService::createUser() creates user with hashed password and default 'customer' role
4. EmailVerificationNotification queued → dispatches VerifyEmail Mailable
5. WelcomeEmail Mailable queued separately
6. User redirected to 'verify email' notice page
7. User clicks verification link → VerifyEmailController marks email_verified_at
8. User redirected to account dashboard

Login Flow

9. User submits login form
10. ThrottleMiddleware checks 5 failed attempts / minute
11. Fortify's LoginResponse validates credentials
12. On success: session regenerated, remember token set if requested
13. TrackUserActivity middleware updates last_active_at
14. Redirect to intended URL or account dashboard

Social Login Flow (Google / Facebook)

15. User clicks 'Continue with Google/Facebook'
16. SocialAuthController::redirect() sends to OAuth provider
17. Provider callback to SocialAuthController::callback()
18. Socialite retrieves provider user data
19. SocialAccountService::findOrCreate() — check social_accounts table
20. If new: create user + social_account record, skip email verification
21. If existing: link to existing account by email match
22. Login user and redirect

7.3 Role & Permission Matrix

Permission Group	Super Admin	Admin	Manager	Customer
View Admin Dashboard	YES	YES	YES	NO

Permission Group	Super Admin	Admin	Manager	Customer
Manage Products	YES	YES	YES (limited)	NO
Manage Orders	YES	YES	YES	Own only
Manage Customers	YES	YES	View only	NO
Manage CMS / Pages	YES	YES	YES	NO
Manage Settings	YES	YES	NO	NO
Manage Roles	YES	NO	NO	NO
View Reports	YES	YES	YES (limited)	NO
Delete Any Record	YES	YES	NO	NO
Impersonate Users	YES	NO	NO	NO

8. Email System

8.1 Email Architecture

All emails are sent via queued Mailable classes using Laravel Queue (Redis). This ensures zero impact on HTTP response times. Templates are Blade views stored in resources/views/emails/ with inline CSS for maximum email client compatibility.

8.2 Transactional Email Templates

Mailable Class	Template	Trigger	Queue
WelcomeEmail	emails.welcome	User registration complete	emails
VerifyEmail	emails.verify	After registration	emails
PasswordResetEmail	emails.password-reset	Forgot password request	emails
OrderConfirmationEmail	emails.order.confirmation	Order placed successfully	orders
OrderShippedEmail	emails.order.shipped	Admin marks order shipped	orders
OrderDeliveredEmail	emails.order.delivered	Admin marks delivered	orders
OrderCancelledEmail	emails.order.cancelled	Order cancelled by user/admin	orders
InvoiceEmail	emails.order.invoice	Invoice generated	orders
NewsletterWelcomeEmail	emails.newsletter.welcome	Newsletter subscription	emails
ContactResponseEmail	emails.contact.response	Admin replies to contact form	emails
LowStockAlertEmail	emails.admin.low-stock	Product stock below threshold	admin
NewOrderAlertEmail	emails.admin.new-order	New order placed	admin

8.3 Email Template Specifications

- Container width: 600px, centered
- Responsive: Media queries for mobile (< 480px, single column)
- CSS: Inline via CSS Inliner (no external stylesheets)
- Font stack: -apple-system, Segoe UI, Arial, sans-serif
- Tested: Gmail (web + app), Outlook 2016/2019/365, Apple Mail, Yahoo Mail
- All images with alt text and hosted on CDN-ready URL
- Unsubscribe link in footer for marketing emails (CAN-SPAM compliance)
- Plain-text version generated automatically via Laravel's Markdown mail

9. Order Processing Flow

9.1 Checkout Pipeline

23. Customer reviews cart → CartService validates stock availability
24. Customer selects/enters shipping address
25. ShippingService calculates available methods and rates
26. Customer applies coupon code → CouponService validates and applies discount
27. TaxService calculates applicable taxes
28. Customer selects payment method
29. CheckoutController::process() fires → wraps in DB::transaction()
30. OrderService::createOrder() creates order record, snapshots product data
31. CartService::clearCart() empties cart
32. PaymentService::processPayment() → gateway-specific handler
33. On payment success: order status set to 'processing'
34. InventoryService::decrementStock() for each ordered item
35. Events fired: OrderPlaced, StockDecremented
36. Listeners: SendOrderConfirmation (queued), SendAdminAlert (queued), UpdateAnalytics
37. Redirect to order success page

9.2 Order Status Lifecycle

Status	Description	Email Triggered
pending	Order created, payment pending	None
payment_failed	Payment declined/failed	None (inline message)
processing	Payment confirmed, being prepared	Order Confirmation Email
on_hold	Awaiting admin review or manual payment	None
shipped	Dispatched to courier	Order Shipped Email
out_for_delivery	With last-mile courier	None
delivered	Confirmed delivered	Order Delivered Email
cancelled	Cancelled by user or admin	Order Cancelled Email
refunded	Refund processed	Refund Confirmation Email
returned	Product returned by customer	None

10. CMS Architecture & Page Builder

10.1 Page Builder System

The CMS uses a component-based page builder where each page consists of ordered sections (page_sections table). Each section has a component_type and a JSON data payload. The frontend renders sections dynamically via @include based on component_type.

10.2 Available Page Builder Components

Component Type	JSON Data Structure	Admin UI Widget
hero_banner	title, subtitle, bg_image, cta_text, cta_url, overlay_color	Image upload + text fields
text_block	content (rich text), alignment, bg_color	TipTap/Quill rich editor
image_block	image, caption, alignment, link, width	Media picker
video_block	source (youtube/vimeo/upload), url/path, autoplay, poster_image	URL input + options
gallery	images[] (path, alt), columns, style (grid/masonry)	Multi-image picker
product_slider	title, source (category/featured/manual), product_ids[], limit	Product selector
product_grid	title, source, product_ids[], columns, limit, show_pagination	Product selector
testimonials	title, ids[] or show_all, style, bg_color	Testimonial picker
faq_section	title, ids[], accordion_style	FAQ picker
cta_section	title, subtitle, button_text, button_url, bg_image, text_color	Text + image fields
custom_html	html (raw HTML/JS)	Code editor (admin-only)
newsletter_form	title, subtitle, placeholder, button_text	Text fields
blog_posts	title, category_id, limit, style	Category selector
divider	style, spacing, color	Simple controls

10.3 CMS Rendering Pipeline

38. Route hits PageController::show($slug)
39. Page model fetched with eager-loaded sections (ordered by sort_order)
40. SEO meta injected via InjectSeoMeta middleware from page record
41. Blade view: resources/views/frontend/pages/show.blade.php
42. @foreach $page->sections → @include 'frontend.components.'. $section->component_type
43. Each component view receives $section->data (decoded JSON)
44. Output cached in Redis with page-level cache tag for instant invalidation on update

11. API Architecture

11.1 REST API Design

The platform exposes a versioned RESTful API at /api/v1/ secured with Laravel Sanctum (token-based). This supports future mobile applications and third-party integrations.

11.2 Key API Endpoints

Method	Endpoint	Description	Auth
GET	/api/v1/products	Product listing with filters	Public
GET	/api/v1/products/{slug}	Single product detail	Public
GET	/api/v1/categories	Category tree	Public
GET	/api/v1/search?q=	Full-text search	Public
POST	/api/v1/auth/register	User registration	Public
POST	/api/v1/auth/login	Get API token	Public
POST	/api/v1/auth/logout	Revoke token	Required
GET	/api/v1/user	Auth user profile	Required
GET	/api/v1/cart	Get cart contents	Optional
POST	/api/v1/cart/items	Add to cart	Optional
PUT	/api/v1/cart/items/{id}	Update cart item	Optional
DELETE	/api/v1/cart/items/{id}	Remove cart item	Optional
POST	/api/v1/checkout	Place order	Required
GET	/api/v1/orders	User order list	Required
GET	/api/v1/orders/{id}	Order details	Required
GET/POST	/api/v1/wishlist	Get / add wishlist	Required
POST	/api/v1/reviews	Submit review	Required
GET	/api/v1/pages/{slug}	CMS page data (JSON)	Public

12. Security Requirements

12.1 Security Checklist

Threat	Mitigation	Laravel Implementation
CSRF Attacks	CSRF token on all state-changing forms	@csrf Blade directive + VerifyCsrfToken middleware
XSS Attacks	Output escaping, Content Security Policy	{{ }} Blade escaping; CSP header middleware
SQL Injection	Parameterised queries exclusively	Eloquent ORM + Query Builder; no raw SQL with user input
Brute Force Login	Rate limiting + lockout	throttle:login middleware; 5 attempts / minute
Mass Assignment	Guarded/fillable on all models	$fillable defined on every Eloquent model
Insecure Direct Object Reference	Policy-based authorization	Laravel Policies + Gates on all resource access
Sensitive Data Exposure	Encrypted storage, HTTPS only	bcrypt passwords; HTTPS enforced; .env secrets
Session Hijacking	Secure, HTTPOnly, SameSite cookies	Session config: secure=true, httponly=true, samesite=strict
File Upload Attacks	Type validation + storage outside webroot	Mime-type check + extension whitelist; stored in storage/
API Abuse	Token auth + rate limiting	Sanctum tokens + throttle:60,1 on API routes
Admin Access	Separate middleware + 2FA (optional)	AdminMiddleware + IP whitelist option
Dependency Vulnerabilities	Regular audits	composer audit + GitHub Dependabot

13. Performance Architecture

13.1 Caching Strategy

Cache Target	Key Pattern	TTL	Invalidation
Homepage data	homepage:data	30 minutes	On product/slider/post update
Product listing page	products:{filters_hash}	15 minutes	On any product change
Product detail	product:{slug}	1 hour	On product update
Category tree	categories:tree	1 day	On category change
CMS Pages	page:{slug}	1 hour	On page save
Settings	settings:all	1 day	On settings save
Navigation menus	menus:{location}	1 day	On menu update
User cart	cart:{session_id}	2 hours	On cart change

13.2 Queue Jobs

Job Class	Queue	Description
SendOrderConfirmationEmail	emails	Order confirmation after checkout
SendOrderStatusEmail	emails	Any order status update email
SendWelcomeEmail	emails	New user welcome email
ProcessProductImages	media	Resize/compress uploaded images into variants
GenerateInvoicePdf	documents	Generate PDF invoice for order
UpdateSearchIndex	search	Sync product changes to Meilisearch
ExportOrdersCsv	exports	Async CSV export for admin reports
SendNewsletterBatch	newsletter	Batch newsletter sends
PruneExpiredCarts	maintenance	Scheduled: delete 24hr+ guest carts
PruneActivityLogs	maintenance	Scheduled: prune logs older than 90 days

13.3 Image Optimisation Pipeline

45. Admin/user uploads image via Media Library
46. ProcessProductImages job dispatched immediately
47. Intervention Image generates variants: thumbnail (150x150), medium (400x400), large (800x800), original
48. WebP versions generated for all variants
49. Original stored in storage/app/public/media/originals/

50. Variants stored in storage/app/public/media/variants/
51. Frontend uses <picture> element with WebP + fallback
52. Lazy loading via loading='lazy' attribute on all product images

14. SEO Strategy

14.1 Technical SEO Implementation

SEO Feature	Implementation
Semantic URLs	product/{slug}, category/{parent}/{child}/{slug} — no IDs in URLs
Canonical Tags	<link rel='canonical'> from page/product/category meta
Meta Tags	Unique meta title, description per page via SEO model columns
Open Graph	og:title, og:description, og:image, og:type on all pages
Twitter Cards	twitter:card, twitter:image on product/blog pages
Structured Data	Product schema (price, availability, reviews), BreadcrumbList, Organization, Article
XML Sitemap	Auto-generated via spatie/laravel-sitemap; products, categories, pages, blog posts
Robots.txt	Dynamic via route; block /admin, /cart, /checkout, /account
Breadcrumbs	Structured breadcrumb on all category/product/blog pages
Page Speed	Redis cache, Vite asset bundling, lazy images, CDN-ready
301 Redirects	Admin-managed redirect table; applied in web middleware
Pagination	rel='prev'/'next' on category listing pages
Hreflang	Multi-language ready via locale prefix in URL

15. Development Roadmap

15.1 Phased Delivery Plan

Phase	Duration	Deliverables
Phase 1: Foundation	Week 1–2	Project setup, Docker environment, database migrations, base models, auth system (register/login/forgot password), role/permission scaffolding, admin layout shell
Phase 2: Product Catalogue	Week 3–4	Category/Brand/Attribute/Product CRUD (admin), product detail page (frontend), product listing with filters, image upload + optimisation pipeline, search integration
Phase 3: Commerce Core	Week 5–6	Cart system, checkout flow (guest + registered), payment gateway integration (Stripe/Razorpay), order management (admin), order history (customer), coupon system
Phase 4: CMS & Pages	Week 7–8	Page builder with all 14 components, blog system, media library, slider management, menu builder, settings module, homepage with all sections
Phase 5: Emails & Notifications	Week 9	All 12 transactional email templates, queue setup with Horizon, in-app notifications, newsletter subscription
Phase 6: SEO & Performance	Week 10	Sitemap generation, structured data, meta management, Redis caching layer, image lazy loading, query optimisation, PageSpeed audit
Phase 7: Customer Account	Week 11	Full account dashboard, address book, wishlist, review system, order tracking, invoice download, social login
Phase 8: Admin Reporting	Week 12	Sales & revenue reports, product performance, customer analytics, CSV/PDF export, activity logs
Phase 9: Testing & QA	Week 13–14	PHPUnit feature tests, browser tests (Laravel Dusk), security audit, cross-browser testing, mobile responsive QA, load testing
Phase 10: Deployment	Week 15	Server provisioning (Nginx + PHP-FPM + MySQL + Redis), CI/CD pipeline, SSL, DNS cutover, performance monitoring setup

16. Deployment Guide

16.1 Server Requirements

- Ubuntu 22.04 LTS (recommended)
- Nginx 1.24+ or Apache 2.4+
- PHP 8.3+ with extensions: BCMath, Ctype, Fileinfo, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML, GD/Imagick, Redis, ZIP
- MySQL 8.0+ or MariaDB 10.6+
- Redis 7.0+
- Node.js 20+ (build only)
- Composer 2.x
- Supervisor (queue workers)
- SSL certificate (Let's Encrypt / paid CA)

16.2 Deployment Steps

53. Server provisioning: install all dependencies above
54. Clone repository to /var/www/jamsora/
55. composer install --no-dev --optimize-autoloader
56. cp .env.example .env && configure all environment variables
57. php artisan key:generate
58. npm install && npm run build
59. php artisan migrate --force
60. php artisan db:seed --class=ProductionSeeder
61. php artisan storage:link
62. php artisan config:cache && route:cache && view:cache && event:cache
63. Configure Nginx virtual host pointing to /public
64. Configure Supervisor for queue:work and horizon
65. Configure cron: * * * * * php artisan schedule:run
66. Set correct file permissions: storage/ and bootstrap/cache/ writable by www-data
67. Configure SSL and force HTTPS in .env and Nginx
68. Point DNS, verify site live, run php artisan telescope:install for monitoring

16.3 Environment Variables (Key)

Variable	Description
APP_ENV=production	Forces production mode (caching, no debug)
APP_DEBUG=false	Disable debug output in production
DB_HOST, DB_DATABASE, DB_USERNAME, DB_PASSWORD	MySQL connection
REDIS_HOST, REDIS_PASSWORD	Redis connection
QUEUE_CONNECTION=redis	Use Redis for job queues
CACHE_STORE=redis	Use Redis for caching

Variable	Description
SESSION_DRIVER=redis	Store sessions in Redis
MAIL_MAILER, MAIL_HOST, MAIL_USERNAME, MAIL_PASSWORD	SMTP configuration
STRIPE_KEY, STRIPE_SECRET / RAZORPAY_KEY, RAZORPAY_SECRET	Payment gateways
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Google OAuth
FACEBOOK_CLIENT_ID, FACEBOOK_CLIENT_SECRET	Facebook OAuth
AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_BUCKET	S3 media storage (optional)
SCOUT_DRIVER=meilisearch, MEILISEARCH_HOST, MEILISEARCH_KEY	Search engine

17. Scalability Recommendations

17.1 Horizontal Scaling Path

- Stateless application: session and cache in Redis (shared across instances)
- Media on S3-compatible storage (not local filesystem) — required for multi-server
- Database read replicas: configure DB_READ_HOST for read operations
- Queue workers: scale independently; 2–10 Supervisor workers per queue
- CDN: Cloudflare or AWS CloudFront in front of Nginx for static assets
- Load balancer: AWS ALB or Nginx upstream block for multiple app servers

17.2 Database Scaling

- Implement read/write splitting via Laravel database::connection('mysql-read')
- Archive old orders (> 2 years) to orders_archive table
- Partition activity_logs table by month
- MySQL query cache + slow query log enabled in production
- Consider Vitess or PlanetScale for extreme scale (10M+ orders)

17.3 Future Architecture Options

- Microservices: extract PaymentService and EmailService as independent services
- Event sourcing: replace order_status_history with full event store
- GraphQL API layer (Lighthouse PHP) for mobile app flexibility
- Push notifications: Laravel WebSockets or Pusher for real-time order updates
- Headless commerce: decouple frontend to Next.js / Nuxt consuming the REST API

18. Database Migration Files Structure

All migrations in database/migrations/, ordered for referential integrity:

Order	Migration File	Creates Table(s)
001	2024_01_01_000001_create_roles_table.php	roles
002	2024_01_01_000002_create_permissions_table.php	permissions, role_permissions, user_permissions
003	2024_01_01_000003_create_users_table.php	users (with role_id FK)
004	2024_01_01_000004_create_social_accounts_table.php	social_accounts
005	2024_01_01_000005_create_otp_verifications_table.php	otp_verifications
006	2024_01_01_000006_create_user_profiles_table.php	user_profiles
007	2024_01_01_000007_create_categories_table.php	categories
008	2024_01_01_000008_create_brands_table.php	brands
009	2024_01_01_000009_create_attributes_table.php	attributes, attribute_values
010	2024_01_01_000010_create_products_table.php	products
011	2024_01_01_000011_create_product_variations_table.php	product_variations, variation_attribute_values
012	2024_01_01_000012_create_product_images_table.php	product_images
013	2024_01_01_000013_create_product_tags_table.php	tags, product_tags, related_products
014	2024_01_01_000014_create_addresses_table.php	addresses
015	2024_01_01_000015_create_carts_table.php	carts, cart_items
016	2024_01_01_000016_create_coupons_table.php	coupons, coupon_usages, coupon_products, coupon_categories
017	2024_01_01_000017_create_shipping_zones_table.php	shipping_zones, shipping_methods
018	2024_01_01_000018_create_tax_rates_table.php	tax_rates
019	2024_01_01_000019_create_payment_methods_table.php	payment_methods

Order	Migration File	Creates Table(s)
020	2024_01_01_000020_create_orders_table.php	orders, order_items, order_addresses
021	2024_01_01_000021_create_order_status_history_table.php	order_status_history
022	2024_01_01_000022_create_shipments_table.php	shipments
023	2024_01_01_000023_create_invoices_table.php	invoices
024	2024_01_01_000024_create_wishlists_table.php	wishlists, wishlist_items
025	2024_01_01_000025_create_reviews_table.php	reviews, review_images
026	2024_01_01_000026_create_notifications_table.php	notifications
027	2024_01_01_000027_create_pages_table.php	pages, page_sections
028	2024_01_01_000028_create_blog_tables.php	blog_categories, blog_posts, blog_tags, blog_post_tags, blog_comments
029	2024_01_01_000029_create_menus_table.php	menus, menu_items
030	2024_01_01_000030_create_sliders_table.php	sliders, slider_slides
031	2024_01_01_000031_create_testimonials_table.php	testimonials, faqs
032	2024_01_01_000032_create_media_table.php	media, media_folders
033	2024_01_01_000033_create_settings_table.php	settings
034	2024_01_01_000034_create_currencies_table.php	currencies
035	2024_01_01_000035_create_activity_logs_table.php	activity_logs
036	2024_01_01_000036_create_newsletter_subscribers_table.php	newsletter_subscribers
037	2024_01_01_000037_create_contact_messages_table.php	contact_messages
038	2024_01_01_000038_add_indexes_to_all_tables.php	Adds all performance indexes

19. Document Revision History

Version	Date	Author	Changes
1.0.0	May 2026	Architecture Team	Initial release — full SRS & Technical Architecture Document

> *This document is the authoritative technical reference for the Jamsora Laravel platform rebuild. All architectural decisions should be validated against this document before implementation.*